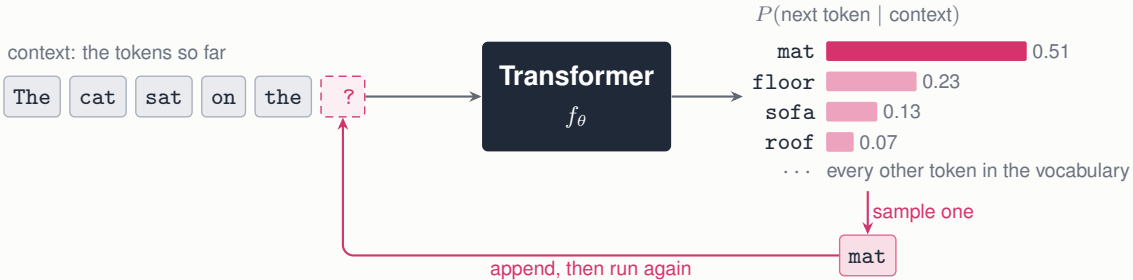
The Transformer, Visually

A component-by-component tour of the architecture inside large language models: tokens, embeddings, attention, MLPs, normalization, and the output head.



An LLM does one thing: predict the next token

Text in, a probability for every token in the vocabulary out. Generation is this step in a loop.



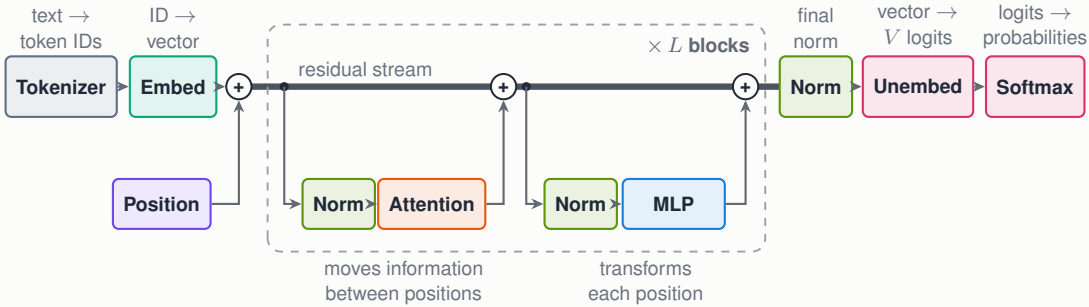
The model is a function: a sequence of tokens goes in, one probability per vocabulary entry comes out.

Everything else is the loop: chat, code and long answers are this one step repeated, each output becoming part of the next input.

probabilities illustrative

The map: a decoder-only transformer

One stream of vectors runs left to right; every component reads from it and adds back to it.



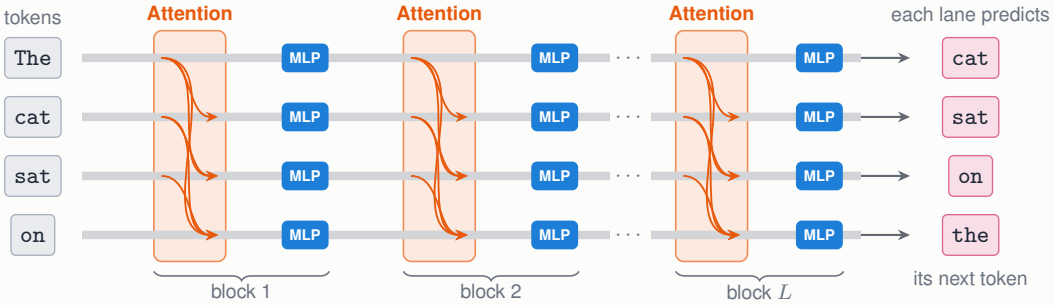
Shapes. T tokens become a $T \times d$ array that keeps its shape through every block; only the last step widens it to $T \times V$.

Two workers. **Attention** is the only place tokens exchange information. The **MLP** processes each token on its own.

Repetition. The same block design is stacked L times (12 to more than 100), each copy with its own weights.

Inside the stream: one lane per token

Attention carries information across lanes, from earlier tokens to later ones. The MLP works within a lane.

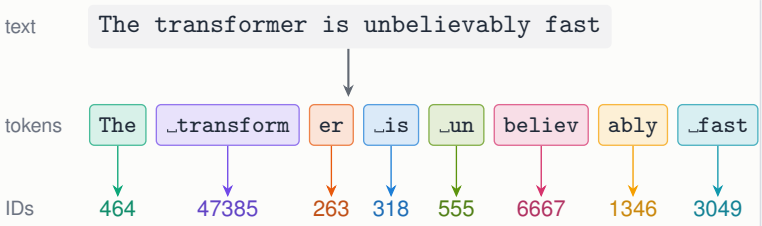


A lane is one position's vector of width d. Every sublayer reads it and adds its result back: $x \leftarrow x + f(x)$.

Only attention crosses lanes, and only forward in the sequence. That one-way flow is the causal structure of an LLM.

Tokenization: text becomes integers

The model never sees characters or words, only IDs of subword pieces from a fixed vocabulary.



Illustrative split and IDs. ␣ marks a leading space, which is part of the token.

How the vocabulary is learned

byte-pair encoding (BPE)

`l` `o` `w` `e` `s` `t`

start from characters

`l` `o` `w` `es` `t`

merge the most frequent pair

`l` `o` `w` `est`

merge again

`low` `est`

. . . until V entries

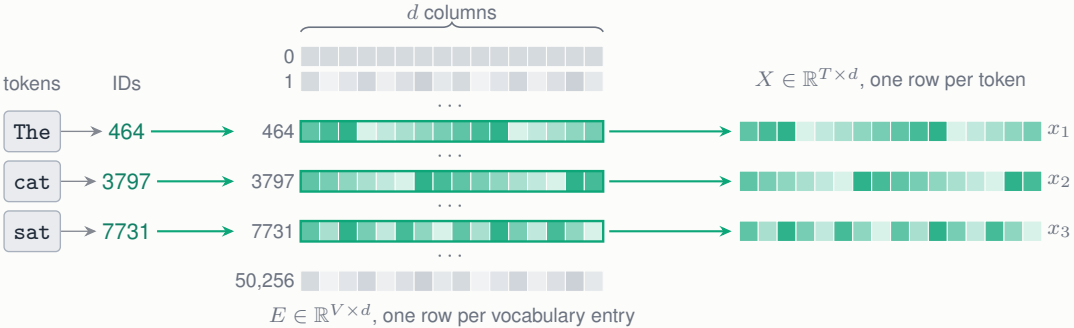
Subwords, not words. Common words are one token; rare words split into pieces, so any string can be encoded.

Vocabulary size V. About 32k to 256k entries in current LLMs; GPT-2 uses 50,257.

Rule of thumb for English: one token is about four characters, or three quarters of a word.

Token embeddings: a lookup table

Each ID selects one row of a learned matrix E. That row is the token's starting vector.

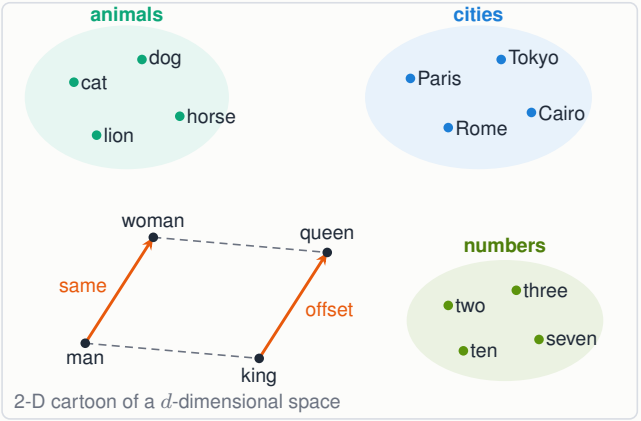


$x_t = E[\mathrm{id}_t]$. No arithmetic, just indexing (equivalent to a one-hot vector times E).

Learned with everything else. GPT-2 small: $50{,}257 \times 768 = 38.6$M parameters, 31% of the model.

What the embedding space looks like

Training arranges tokens so that geometry reflects usage: neighbors are similar, directions are relations.



2-D cartoon of a d-dimensional space

Neighbors. Tokens used in similar contexts end up with similar vectors.

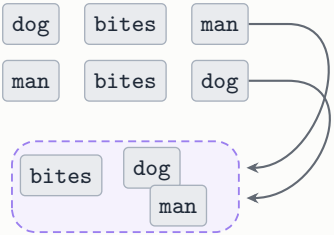
Directions. Consistent offsets can encode a relation:
king $-$ *man* $+$ *woman* $\approx$ *queen*
(word2vec, Mikolov et al. 2013).

Not yet contextual. "bank" gets the same input vector by a river or in finance. The layers above resolve it.

Width d. 768 in GPT-2 small; 4,096 to 16,384 in current large models.

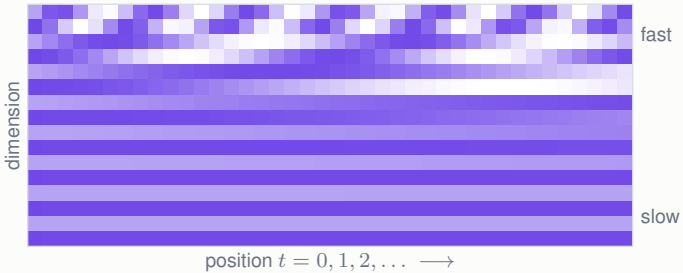
Position: telling the model what came first

Attention treats its input as a set. Order has to be injected explicitly.



Without position information, both sentences are the same bag of vectors.

Sinusoidal encoding: a unique pattern per position



$$PE_{t,2i} = \sin(t\,\omega_i), \qquad PE_{t,2i+1} = \cos(t\,\omega_i), \qquad \omega_i = 10000^{-2i/d}$$

Fixed sinusoids, added to the embedding (original Transformer, 2017).

Learned table $P[t]$, added the same way (BERT, GPT-2).

Rotary (RoPE), applied inside attention. Next slide.

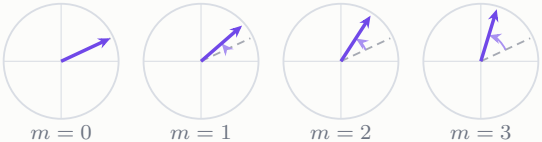
Rotary position embedding (RoPE)

Encode position as a rotation. Attention scores then depend only on how far apart two tokens are.

a fast pair: large θ_i



a slow pair: small θ_i



Each pair of dimensions is a clock hand. At position m it has turned by $m\,\theta_i$, with $\theta_i = 10000^{-2i/d}$. Nothing is learned.

Rotate queries and keys by their positions:

$$\tilde{q}_m = R(m\theta)\,q \qquad \tilde{k}_n = R(n\theta)\,k$$

Their dot product only sees the difference:

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = \langle R\big((m-n)\theta\big)\,q,\ k \rangle$$

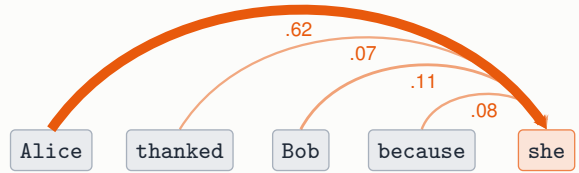
Relative by construction. Shift both tokens by the same amount and the score is unchanged.

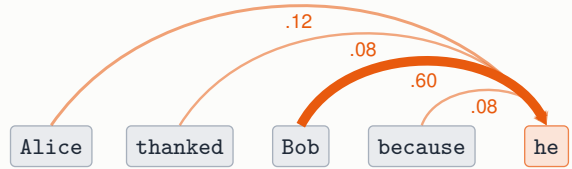
Where it acts. On q and k in every attention layer. Values and the residual stream are untouched.

In use. Llama, Mistral, Qwen and Gemma families (Su et al. 2021).

Attention: each token gathers what it needs from the past

A weighted average over earlier positions, with weights computed from the content of the tokens.





The task. To continue the sentence after “she”, the model has to know who “she” is.

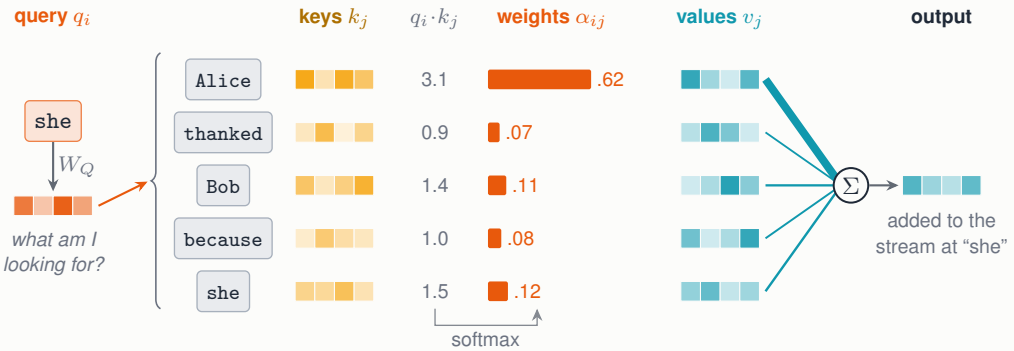
The mechanism. The position holding “she” pulls in information from earlier positions. Arrow width is the attention weight.

Content decides. Change one word (she to he) and the weights re-route to Bob. The parameters are the same; only the data changed.

weights illustrative

Queries, keys and values

A soft dictionary lookup: compare one query with every key, then blend the values in proportion to the match.



Query $q_i = W_Q\, x_i$
What am I looking for?

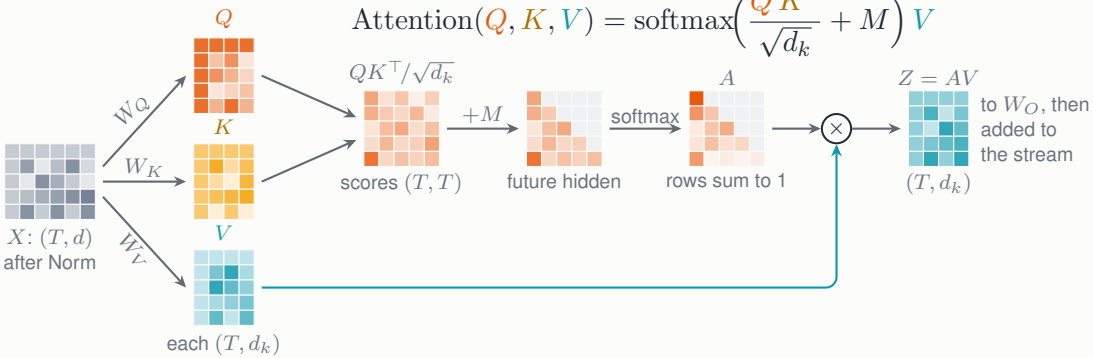
Key $k_j = W_K\, x_j$
What do I contain?

Value $v_j = W_V\, x_j$
What do I hand over if chosen?

Scaled dot-product attention, in matrices

All T queries are handled at once: three projections, one matrix product, a mask, a softmax, one more product.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q\,K^{\top}}{\sqrt{d_k}} + M\right) V$$

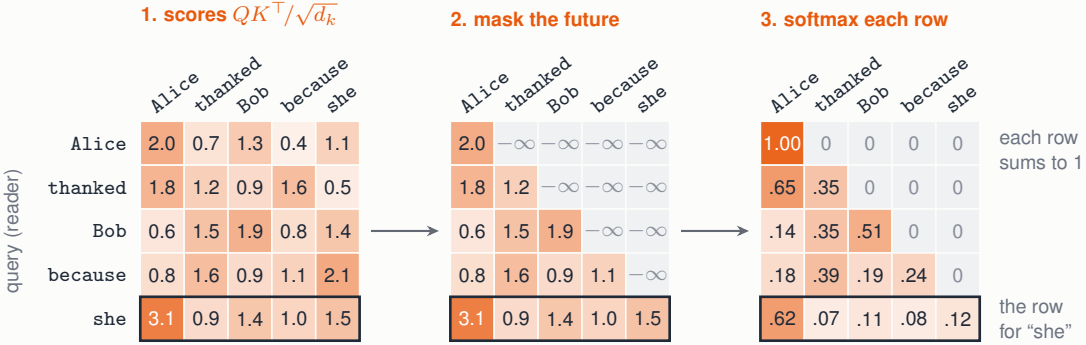


Why $\sqrt{d_k}$. Unscaled dot products grow with d_k and saturate the softmax, shrinking its gradients (the paper's stated motivation).

Why it is fast. Everything is dense matrix algebra, so all positions are processed in parallel on a GPU.

From scores to weights: a worked example

Row i of the matrix is token i's query. After masking and softmax it is a probability distribution over what it may read.

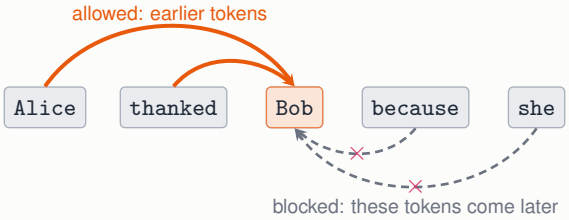


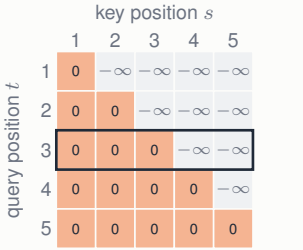
Mask. Adding $-\infty$ above the diagonal makes those weights exactly zero after the softmax.

Read the last row. “she” puts 0.62 of its attention on “Alice”: the weights of the previous slides.

The causal mask

Position t may read positions $1, \ldots, t$ and nothing later. This single constraint makes the model autoregressive.





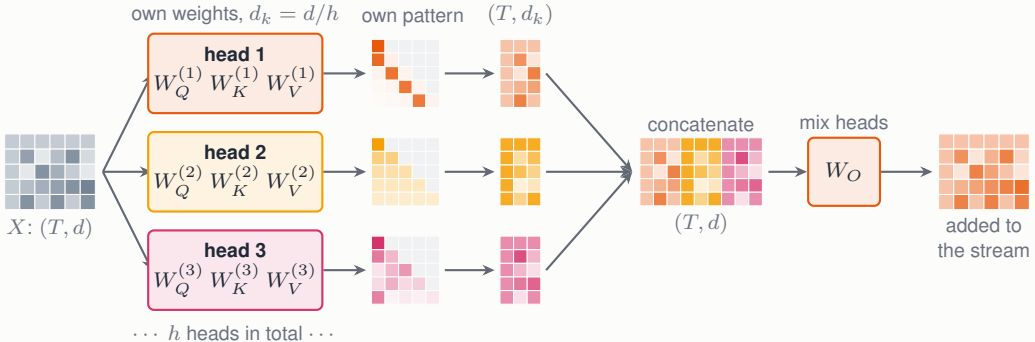
$M_{ts} = 0$ if $s \leq t$, otherwise $-\infty$

Training. One forward pass yields T next-token predictions, and none of them can see its own answer.

Inference. Earlier positions never depend on later ones, so their keys and values are computed once and cached.

Multi-head attention

Several small attention mechanisms run side by side, each free to track a different relationship.



Parallel views. One head can follow syntax while another tracks a name.

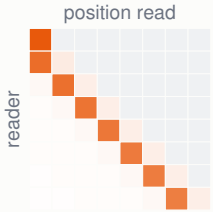
Same cost. h heads of width d/h cost about as much as one head of width d.

GPT-2 small. $d = 768$, $h = 12$, so each head works in $d_k = 64$ dimensions.

What individual heads learn to do

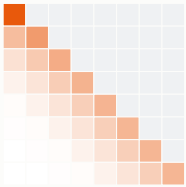
Idealized attention patterns reported in interpretability studies. Rows are readers, columns are the positions they read.

Previous token



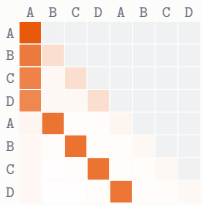
Each position reads the one just before it.

Local window



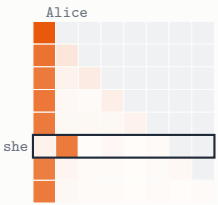
A decaying blend of the last few tokens.

Induction



On a repeat, read what followed last time and copy it.

Reference

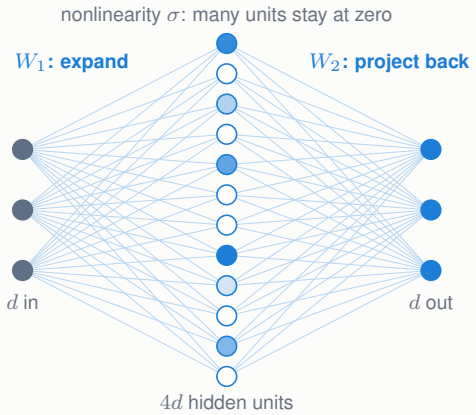


A pronoun reads its antecedent; idle rows park on the first token.

Sources: Clark et al. 2019 (BERT heads for syntax and coreference); Elhage et al. 2021 and Olsson et al. 2022 (previous-token and induction heads). Real heads are messier, and many have no single clean role.

The MLP: where each token is transformed

A two-layer network applied to every position separately: widen, apply a nonlinearity, project back.



$$\mathrm{MLP}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$$

Position-wise. The same weights are applied at every position. No information moves between tokens here.

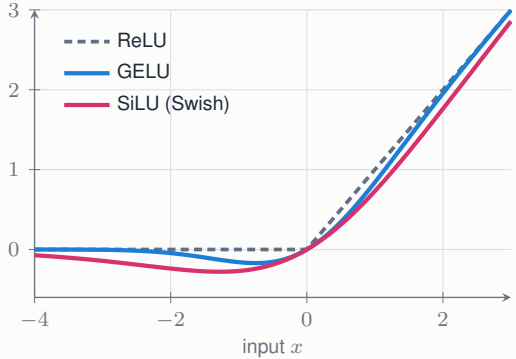
Wide in the middle. $d_{\text{ff}} = 4d$; in GPT-2 small, $768 \to 3072 \to 768$.

Most of the weights. $8d^2$ per block against $4d^2$ for attention: about two thirds.

One reading: a key–value memory. Rows of W_1 detect a pattern in the stream; the matching columns of W_2 write the associated update (Geva et al. 2021).

Activation functions

The nonlinearity between the two matrices. Without it, the whole MLP would collapse into one linear map.



GELU and SiLU are smooth and dip slightly below zero, so small negative inputs still pass some gradient.

ReLU $\max(0, x)$
The original Transformer (2017).

GELU $x\,\Phi(x)$, with Φ the normal CDF
BERT, GPT-2.

SiLU $x\,\mathrm{sigmoid}(x)$
The gate inside SwiGLU.

SwiGLU, the gated MLP of Llama and Mistral:

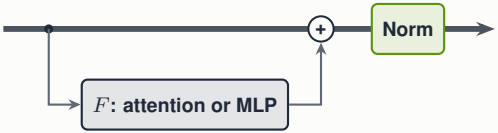
$$\mathrm{MLP}(x) = W_2\big(\mathrm{SiLU}(W_1 x) \odot W_3 x\big)$$

Three matrices with hidden width $\frac{8}{3}d$ keep the same $8d^2$ weight budget (Shazeer 2020).

Residual connections and where the norm goes

Each sublayer adds a correction to its input. Norm placement decides how clean the identity path is.

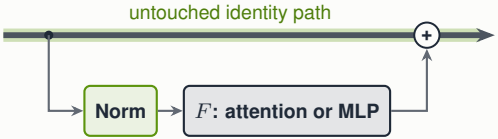
Post-norm
original, 2017



$$x_{l+1} = \mathrm{Norm}\big(x_l + F(x_l)\big)$$

the norm sits on the main path

Pre-norm
GPT-2 onward;
the standard today



$$x_{l+1} = x_l + F\big(\mathrm{Norm}(x_l)\big)$$

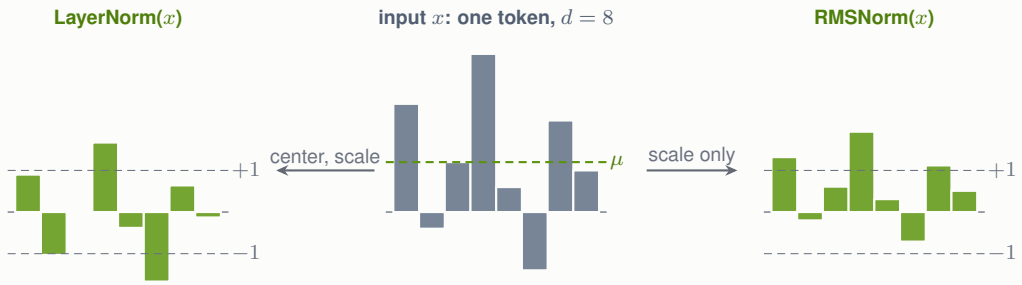
the norm sits inside the branch

Why residuals. Signal and gradient have a direct route through all L layers, which is what makes very deep stacks trainable (He et al. 2016).

Why pre-norm. More stable at depth, with little or no learning-rate warm-up (Xiong et al. 2020). It needs one final norm before the output head.

LayerNorm and RMSNorm

Rescale each token's vector to a standard size, using only that token's own d features.



$$\gamma \odot \frac{x - \mu}{\sigma} + \beta$$

Subtract the mean, divide by the standard deviation; learned gain and bias.

$\mu = 1.2,\ \sigma = 1.56,\ \text{RMS} = 1.97$

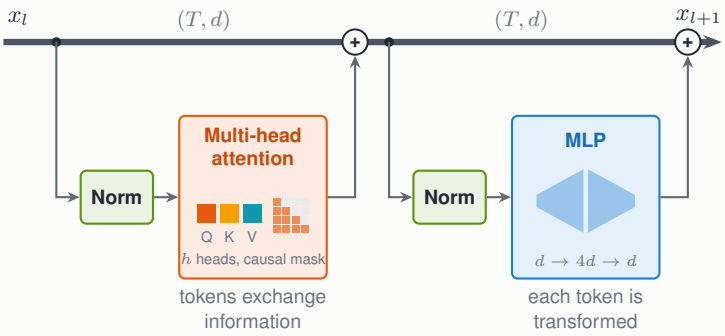
Shown with $\gamma = 1$, $\beta = 0$. Statistics come from a single token, so nothing depends on the batch or on other positions.

$$\gamma \odot \frac{x}{\text{RMS}(x)}$$

No centering, no bias, less work. Llama, Mistral, Qwen, Gemma.

One complete transformer block

Two sublayers, each wrapped as norm, compute, add. The output has the same shape as the input.



$$h = x_l + \mathrm{Attn}(\mathrm{Norm}(x_l))$$

$$x_{l+1} = h + \mathrm{MLP}(\mathrm{Norm}(h))$$

Per block: about $12\,d^2$ weights

$4d^2$ in attention (W_Q, W_K, W_V, W_O)

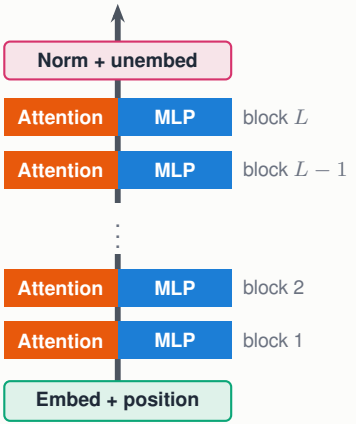
$8d^2$ in the MLP (W_1, W_2)

GPT-2 small: 7.1M per block, 12 blocks.

Nothing here depends on T. The same weights serve any sequence length, and same-shape input and output is what lets blocks stack.

Stacking blocks: depth and width

The same block repeated L times with separate weights. Later blocks build on what earlier blocks wrote to the stream.



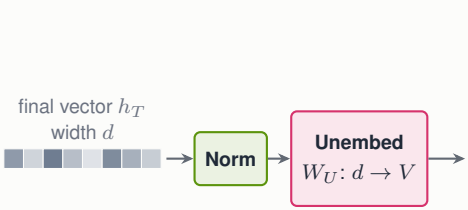
Model	Layers L	Width d	Heads	Parameters
GPT-2 small (2019)	12	768	12	124M
GPT-2 XL (2019)	48	1,600	25	1.56B
GPT-3 davinci (2020)	96	12,288	96	175B
Llama-2 7B (2023)	32	4,096	32	6.7B
Llama-3 70B (2024)	80	8,192	64	70.6B
Llama-3.1 405B (2024)	126	16,384	128	406B

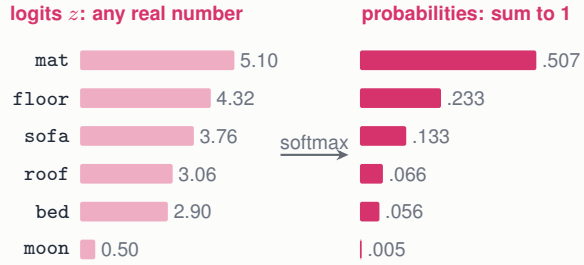
Scaling rule. Block weights grow as $12\,L\,d^2$: doubling the depth doubles them, doubling the width quadruples them.

Division of labor (a tendency, from probing studies): earlier layers carry more local and syntactic features, later layers more abstract ones (Tenney et al. 2019).

The output head: from a vector to a distribution

The final vector at the last position is compared with every vocabulary entry, then normalized.





six-token toy vocabulary; a real model scores all V entries

$$z = W_U \,\mathrm{Norm}(h_T) \;\in\; \mathbb{R}^V$$

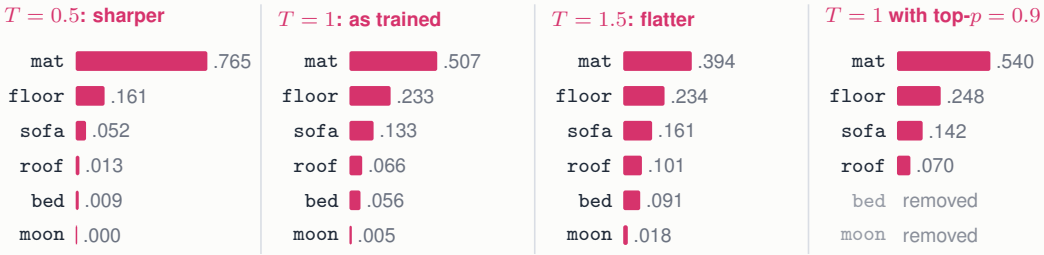
$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

One logit per token. z_i is the dot product of the final vector with token i's row of W_U.

Weight tying. GPT-2 and others reuse the embedding matrix: $W_U = E^\top$ (Press and Wolf 2017).

Choosing the next token: temperature, top-k, top-p

The model's work ends at the distribution. How to draw from it is a separate, tunable choice.



Temperature. Divide the logits by T before the softmax. $T < 1$ sharpens, $T > 1$ flattens, $T \to 0$ is greedy (always the top token).

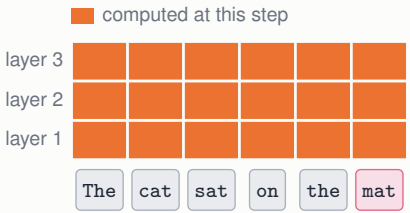
Top-k. Keep the k most likely tokens, renormalize, sample. A fixed-size shortlist.

Top-p (nucleus). Keep the smallest set whose probabilities add up to p. The shortlist adapts to how confident the model is (Holtzman et al. 2020).

Generation and the KV cache

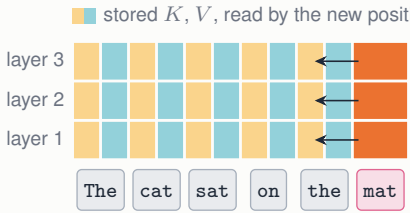
Because of the causal mask, earlier positions never change. Their keys and values can be stored and reused.

Without a cache: every position is recomputed



per layer, at step t: $12\,t\,d^2 + 2\,t^2 d$

With a cache: only the new position is computed



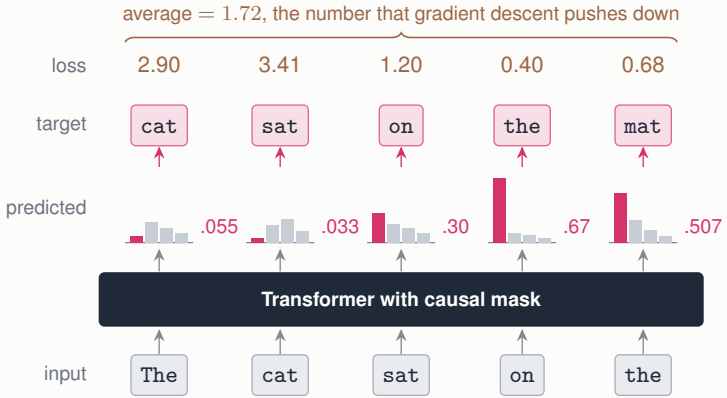
per layer, at step t: $12\,d^2 + 2\,t\,d$

Two phases. Prefill runs the whole prompt in parallel and fills the cache. Decode then adds one token per forward pass.

Cache size: $2\,L\,T\,d$ numbers. With $L = 32$, $d = 4096$ (Llama-2 7B scale), $T = 4096$ and 16-bit floats: 2 GiB per sequence.

Training: predict every next token at once

The text supplies its own labels. One pass over a sequence of T tokens produces T predictions and T losses.



pink bar: probability given to the correct next token; loss $= -\ln$ of it (illustrative values)

$$\mathcal{L} = -\frac{1}{T}\sum_t \log p_\theta(x_{t+1} \mid x_{\le t})$$

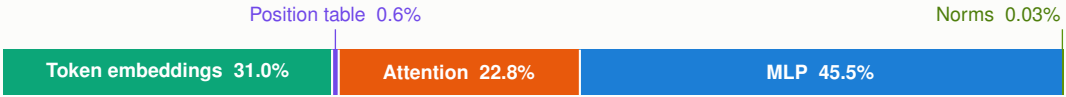
Shifted targets. The target at position t is the input at $t+1$.

No peeking. The causal mask hides the answer from each position.

Scale. Llama 3: more than 15 trillion training tokens.

Where the parameters live: GPT-2 small

124,439,808 weights, counted from the architecture. The MLPs hold the largest share.



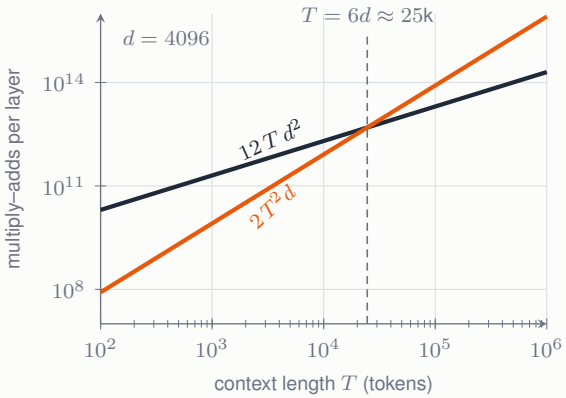
Component	Count	In GPT-2 small	Parameters
Token embedding E	$V\,d$	$50{,}257 \times 768$	38,597,376
Position table	$T_{\max}\,d$	$1{,}024 \times 768$	786,432
Attention	$4d^2 + 4d$ per block	$\times$ 12 blocks	28,348,416
MLP	$8d^2 + 5d$ per block	$\times$ 12 blocks	56,669,184
Norm gains and biases	$2d$ per norm	$\times$ 25 norms	38,400
Unembedding	tied to E		0
Total			**124,439,808**

Small model. The vocabulary table alone is almost a third of the weights.

Large model. Blocks dominate. $12\,L\,d^2$ with $L = 32$, $d = 4096$ gives 6.4B; Llama-2 7B has 6.7B in total.

Compute cost: linear in the weights, quadratic in the context

Per layer, the matrix products scale with T and the attention pattern with T^2. The two cross at $T = 6d$.



$12\,T\,d^2$: weight products. The Q, K, V, O projections and the MLP. Linear in T.

$2\,T^2 d$: scores and mixing. Every token against every earlier token. Quadratic in T.

Memory. The $T \times T$ attention matrix, which FlashAttention avoids storing by computing it in tiles, and the KV cache.

Decode. Each new token touches every weight once, so single-stream generation is usually limited by memory bandwidth.

The family, and what changed since 2017

Three ways to wire the same block, and the refinements that separate current LLMs from the original design.



Encoder-only. BERT. Every token sees both directions. Embeddings, classification.



Encoder–decoder. The 2017 Transformer, T5. The decoder also attends to the encoder. Translation.



Decoder-only. GPT, Llama, Mistral, Qwen, Gemma. Causal attention; this deck.

icons: which positions each token may read

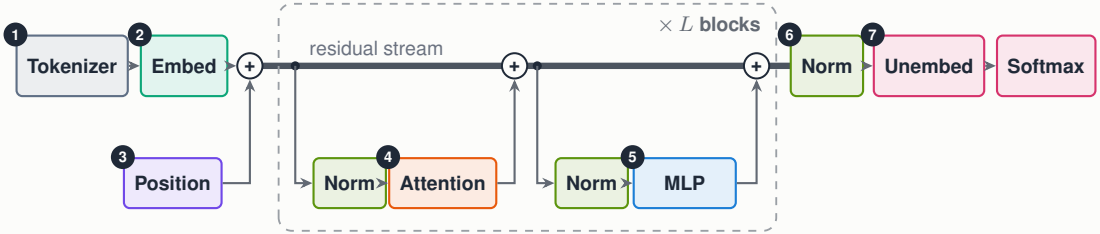
	2017 original	Typical LLM today
Layout	encoder–decoder	decoder-only
Norm	LayerNorm, after	RMSNorm, before
Position	sinusoids, added	RoPE, in attention
Activation	ReLU	SwiGLU or GELU
Attention	multi-head	grouped-query

Grouped-query attention. Several query heads share one key/value head, shrinking the KV cache. Llama-3 70B: 64 query heads, 8 key/value heads.

Unchanged: the residual stream, dot-product attention and the MLP.

The whole machine on one slide

Seven components, one loop: tokens in, a distribution out, sample, append, repeat.



1 **Tokenizer.** Text to subword IDs.

2 **Embedding.** Each ID to a vector of width d.

3 **Position.** Order, added at the input or rotated into q, k.

4 **Attention.** Each token reads earlier tokens: Q, K, V, h heads.

5 **MLP.** Per-token transform; most of the weights.

6 **Norm and residual.** Stable scale; every sublayer adds to the stream.

7 **Unembed and softmax.** Final vector to a distribution over V tokens.